virtspawn

REST API Reference

Complete reference for all 30+ API endpoints

Base URL: http://localhost:8081/api/v1

All responses are JSON | Errors return {"error": "message"}

GET POST DELETE

Content-Type: application/json

https://github.com/ssahani/-virtspawn

Virtual Machine Endpoints

VM Lifecycle

Method	Endpoint	Description	Request Body
GET	/vms	List all VMs	-
GET	/vms/{name}	Get VM details	-
GET	/vms/{name}/xml	Get VM XML definition	-
POST	/vms	Create new VM	{name, vcpus, memory_mb, disk_gb}
POST	/vms/{name}/start	Start VM	-
POST	/vms/{name}/stop	Force stop VM	-
POST	/vms/{name}/shutdown	Graceful shutdown	-
POST	/vms/{name}/reboot	Reboot VM	-
POST	/vms/{name}/pause	Pause VM	-
POST	/vms/{name}/resume	Resume paused VM	-
DELETE	/vms/{name}	Delete VM	-
POST	/vms/{name}/clone	Clone VM	{new_name}
POST	/vms/{name}/rename	Rename VM (must be off)	{new_name}
POST	/vms/{name}/autostart/{bool}	Set autostart	-
POST	/vms/{name}/vcpus/{n}	Set vCPU count	-
POST	/vms/{name}/memory/{mb}	Set memory (MB)	-

VM API Examples

List VMs

```
$ curl -s http://localhost8081/api/v1/vms

Response:

[{"name":"myvm","state":"running",
 "vcpus":4,"memory_mb":4096}]
```

Create VM

```
$ curl -X POST http://localhost8081/api/v1/vms \
 -H "Content-Type: application/json" \
 -d '{"name":"web-server",
    "vcpus":4, "memory_mb":8192,
    "disk_gb":40, "network":"default"}'
```

Get VM Details

```
$ curl -s localhost8081/api/v1/vms/myvm

Response includes:
name, uuid, state, vcpus, memory_mb,
os_type, arch, autostart, persistent,
interfaces[], disks[]
```

VM Lifecycle

```
# Start
$ curl -X POST localhost8081/api/v1/vms/myvm/start

# Graceful shutdown
$ curl -X POST localhost8081/api/v1/vms/myvm/shutdown

# Force stop
$ curl -X POST localhost8081/api/v1/vms/myvm/stop
```

Clone & Resize

```
$ curl -X POST localhost8081/api/v1/vms/myvm/clone \
 -d '{"new_name":"myvm-clone"}'
$ curl -X POST localhost8081/api/v1/vms/myvm/vcpus/8
```

Delete VM

```
$ curl -X DELETE localhost8081/api/v1/vms/myvm

{"status":"deleted","name":"myvm"}
```

Network & Storage Endpoints

Networks

Method	Endpoint	Description	Request Body
GET	/networks	List all networks	-
GET	/networks/{name}/xml	Get network XML	-
POST	/networks	Create network	{name, subnet, dhcp_start, dhcp_end}
POST	/networks/{name}/start	Start network	-
POST	/networks/{name}/stop	Stop network	-
POST	/networks/{name}/autostart/{bool}	Set autostart	-
DELETE	/networks/{name}	Delete network	-

Storage

Method	Endpoint	Description	Request Body
GET	/storage/pools	List storage pools	-
POST	/storage/pools	Create pool	{name, pool_type, target_path}
DELETE	/storage/pools/{name}	Delete pool	-
GET	/storage/pools/{name}/xml	Get pool XML	-
GET	/storage/pools/{p}/volumes	List volumes in pool	-
POST	/storage/pools/{p}/volumes	Create volume	{name, capacity_gb, format}
DELETE	/storage/pools/{p}/volumes/{v}	Delete volume	-
POST	/storage/pools/{p}/volumes/{v}/resize	Resize volume	{capacity_gb}
POST	/storage/pools/{p}/volumes/{v}/clone	Clone volume	{new_name}

Snapshots, Metrics & Node Endpoints

Snapshots

Method	Endpoint	Description	Request Body
GET	/snapshots	List all snapshots	-
GET	/vms/{name}/snapshots	List snapshots for VM	-
POST	/vms/{name}/snapshots	Create snapshot	{name, description}
POST	/vms/{n}/snapshots/{s}/revert	Revert to snapshot	-
DELETE	/vms/{n}/snapshots/{s}	Delete snapshot	-

Metrics & Node

Method	Endpoint	Description	Request Body
GET	/metrics	All VM metrics	-
GET	/metrics/{name}	Single VM metrics	-
GET	/node	Host node info	-
GET	/health	Health check	-
GET	/templates	List VM templates	-

WebSocket

Method	Endpoint	Description	Request Body
WS	/ws/v1/watch	Real-time VM state changes	subscribe
WS	/ws/v1/console/{name}	Serial console I/O	bidirectional
WS	/ws/v1/vnc/{name}	VNC proxy (binary)	bidirectional

Metrics response: cpu_time_ns, memory_total_mb, memory_used_mb, memory_pct, disk/net I/O

Advanced Endpoints

Guest Agent & Boot

Method	Endpoint	Description	Request Body
GET	/vms/{name}/interfaces	Guest IP addresses (via agent)	-
GET	/vms/{name}/hostname	Guest hostname	-
GET	/vms/{name}/boot	Get boot configuration	-
POST	/vms/{name}/boot	Set boot order	{devices: ["hd","cdrom"]}

CD-ROM, Save & Migration

Method	Endpoint	Description	Request Body
POST	/vms/{name}/cdrom/insert	Insert CD-ROM ISO	{iso_path, target}
POST	/vms/{name}/cdrom/eject/{target}	Eject CD-ROM	-
POST	/vms/{name}/managed-save	Managed save (hibernate)	-
DELETE	/vms/{name}/managed-save	Remove saved state	-
GET	/vms/{name}/managed-save/status	Check if save exists	-
POST	/vms/{name}/migrate	Migrate VM	{dest_uri, live}
POST	/vms/{name}/balloon/{mb}	Memory balloon	-

Infrastructure

Method	Endpoint	Description	Request Body
GET	/capabilities	Hypervisor capabilities	-
GET	/sysinfo	System BIOS/SMBIOS info (XML)	-
GET	/devices	List node devices	?capability=pci|net|usb
GET	/devices/{name}	Get device XML	-
GET	/nwfilters	List network filters	-
DELETE	/nwfilters/{name}	Delete network filter	-
GET	/secrets	List secrets	-
DELETE	/secrets/{uuid}	Delete secret	-

Response Formats

VM Details Response

```
{
  "name": "web-server",
  "uuid": "a1b2c3d4-...",
  "state": "running",
  "vcpus": 4,
  "memory_mb": 8192,
  "os_type": "hvm",
  "arch": "x86_64",
  "autostart": true,
  "persistent": true,
  "interfaces": [
    {"mac_address":"52:54:00:...",
     "source":"default","model":"virtio"}
  ],
  "disks": [
    {"device":"disk","source":"/var/...",
     "driver":"qcow2","target":"vda"}
  ]
}
```

Node Info Response

```
{"hostname":"server1",
"hypervisor":"QEMU",
"hypervisor_version":"10.1.4",
"lib_version":"11.6.0",
"cpu_model":"x86_64",
"cpu_cores":14,"memory_mb":31524}
```

Metrics Response

```
{
  "name": "web-server",
  "cpu_time_ns": 58230000000,
  "vcpus": 4,
  "memory_total_mb": 8192,
  "memory_used_mb": 3421,
  "memory_pct": 41.8,
  "disk_rd_bytes": 1048576,
  "disk_wr_bytes": 524288,
  "net_rx_bytes": 204800,
  "net_tx_bytes": 102400
}
```

Error Response

```
# 404 Not Found
{"error": "VM 'myvm' not found: ..."}

# 500 Internal Server Error
{"error": "Failed to start VM: ..."}

# Validation Error
{"error": "capacity_gb must be > 0"}
```

WebSocket API

VM State Watch `ws://host:8081/ws/v1/watch`

```
# Heartbeat (every 2s)
{"event":"heartbeat","vm_count":5}

# State change
{"event":"changes","changes":[
 {"event":"state_change",
  "name":"myvm",
  "old_state":"shutoff",
  "new_state":"running"}
]}

# VM added/removed
{"event":"changes","changes":[
 {"event":"vm_added","name":"new-vm",
  "state":"running"}
]}
```

Serial Console `ws://host:8081/ws/v1/console/{name}`

```
# Bidirectional text I/O
# Send: keystrokes as text frames
# Receive: terminal output as text frames

# Connection example (JavaScript):
const ws = new WebSocket(
  "ws://localhost:8081/ws/v1/console/myvm"
);
ws.onmessage = (e) => term.write(e.data);
term.onData((d) => ws.send(d));
```

VNC Proxy `ws://host:8081/ws/v1/vnc/{name}`

```
# Binary WebSocket proxy to VM's VNC port. Used by noVNC in the Web UI.
# Send/receive: binary VNC protocol frames. Connects to qemu VNC on 127.0.0.1:{port}
```

All WebSocket connections auto-close when the client disconnects or the VM stops

Status Codes & Error Handling

200	OK	Successful GET/POST/DELETE
404	Not Found	VM, network, pool, volume, or snapshot not found
500	Internal Server Error	Libvirt operation failed, connection error

Input Validation

- VM/resource names: alphanumeric, dash, underscore, dot (1-64 chars)
- vCPUs: 1-256
- Memory: 64 MB - 1 TB (1,048,576 MB)
- Disk size: 1 GB - 10 TB (10,240 GB)
- Migration URI: must start with qemu://, qemu+ssh://, qemu+tcp://, qemu+tls://, qemu+unix://
- Volume capacity: must be > 0, checked_mul for overflow protection
- ISO paths: must be absolute, canonicalized, and point to existing file
- Network IPs: validated as IPv4 format, subnet as 3-octet prefix
- Memory balloon: validated against 64 MB - 1 TB range

Security: No CORS headers (same-origin only) | XML-escaped inputs | PTY path validated | Audit logged

Automation Examples

Bash: Bulk create VMs

```bash
#!/bin/bash
API="http://localhost:8081/api/v1"
for i in $(seq 1 5); do
  curl -s -X POST "$API/vms" \
    -H "Content-Type: application/json" \
    -d "{
      \"name\": \"worker-$i\",
      \"vcpus\": 2,
      \"memory_mb\": 4096,
      \"disk_gb\": 20
    }"
  curl -s -X POST "$API/vms/worker-$i/start"
  echo "Created and started worker-$i"
done
```

Python: Monitor VMs

```python
import requests, time

API = "http://localhost:8081/api/v1"

while True:
    vms = requests.get(f"{API}/vms").json()
    metrics = requests.get(f"{API}/metrics").json()
    for m in metrics:
        if m["memory_pct"] > 90:
            print(f"ALERT: {m['name']} at
                {m['memory_pct']}% memory")
    time.sleep(30)
```

Snapshot backup script

```bash
API="http://localhost:8081/api/v1"
DATE=$(date +%Y%m%d)
for vm in $(curl -s $API/vms | jq -r ".[].name"); do
  curl -s -X POST "$API/vms/$vm/snapshots" \
    -d "{\"name\":\"backup-$DATE\"}"
done
```

Health check (Nagios/Zabbix)

```bash
#!/bin/bash
HEALTH=$(curl -sf localhost:8081/api/v1/health)
if [ $? -ne 0 ]; then
  echo "CRITICAL: virtspawn daemon down"
  exit 2
fi
echo "OK $HEALTH"
```